

• PRODUCT LAUNCH • 2026

Nimbus — *background jobs*, finally boring

Durable task execution for product teams who would rather not babysit a queue. Submit work, get results, sleep through the night.

• WHY NOW

99.98%

of jobs land on the *first try* — no dead-letter archaeology, no 3 a.m. replays.

Three promises, *no asterisks*

Durable

Every job is persisted before it runs. A crash mid-flight resumes exactly where it stopped — never silently dropped.

Observable

One timeline per job: inputs, retries, logs, and the final result, all queryable from the dashboard or the API.

Boring

No brokers to tune, no shards to rebalance. You ship the handler; we run the infrastructure underneath it.

• PICK A PLAN

Sized for *how you run*

Starter

10k jobs / month. Shared workers, 7-day history. Ideal for side projects and early prototypes.

Team

5M jobs / month. Dedicated workers, 30-day history, priority queues, and on-call alerting.

Scale

Unmetered jobs. Multi-region execution, 1-year history, custom retention, and a named reliability engineer.

A small *vocabulary*

Job

One unit of work with inputs and a handler.

Run

A single attempt at a job; retries make new runs.

Queue

An ordered, durable lane jobs flow through.

Handler

Your function that turns inputs into a result.

- WHAT SHIPS IN THE BOX

The starter *project*

nimbus.toml Project
+ queue config.

handlers/ Your job
functions live here.

schemas/ Typed
inputs and results.

tests/ Replay fixtures
for every job.

- TWO HALVES OF THE SAME IDEA

Submit once, *trust it*

At submit time

The SDK validates inputs against your schema, assigns a stable job id, and persists the request before returning — so a network blip can't lose work.

At run time

A worker leases the job, runs your handler in isolation, and commits the result atomically. Crash, retry, or scale-out, the job advances exactly once.

Three roads, *one destination*

From cron

Scripts that drifted into critical infrastructure. Nimbus gives them retries, history, and alerting without a rewrite.

From a broker

Raw queues that need babysitting. Keep the mental model; drop the operational tax of brokers and consumers.

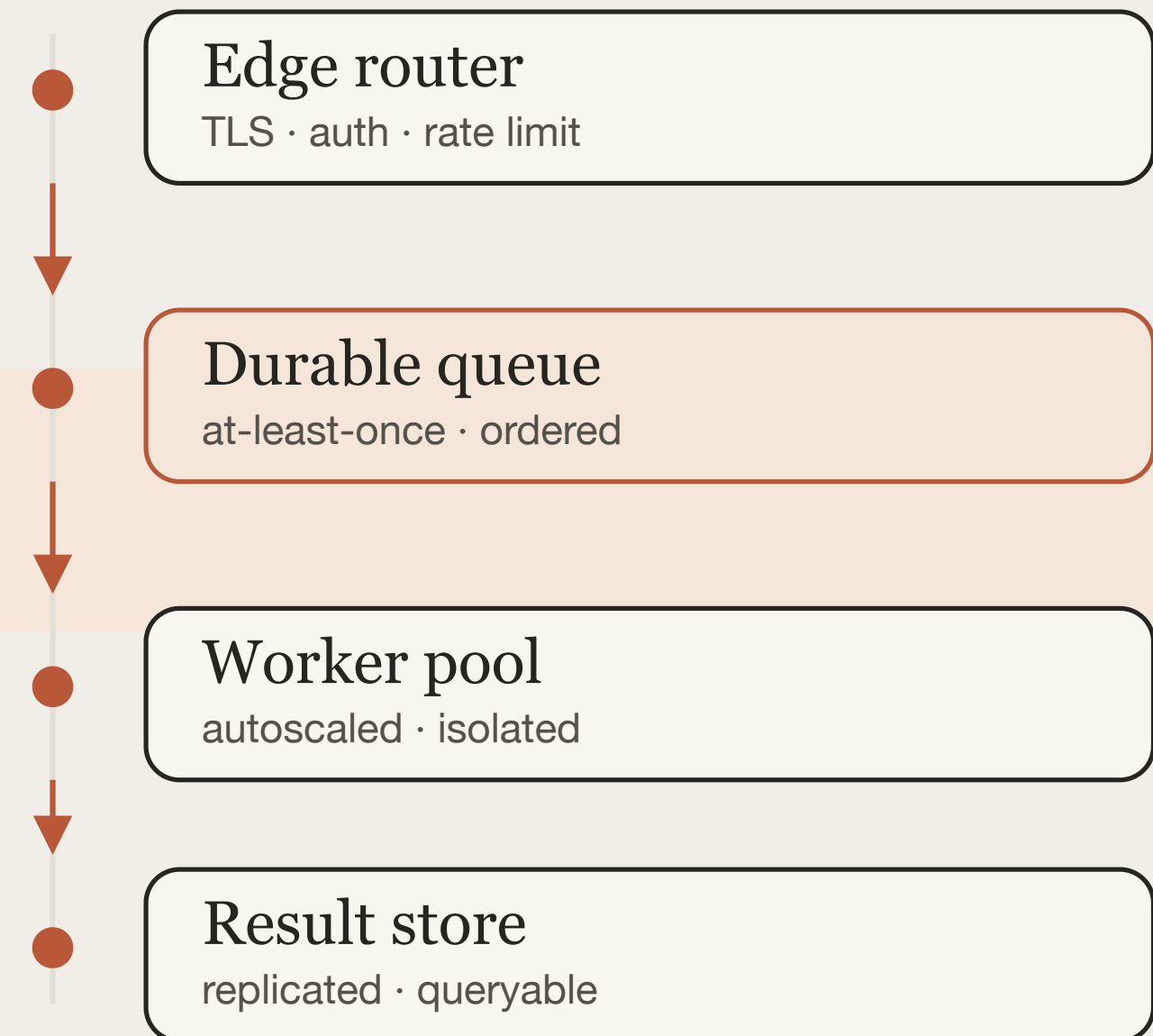
From a SaaS

A black-box runner you've outgrown. Bring handlers in-house with replayable history and no per-task surcharge.

A request's *short life*

Every submission walks the same four stops. The edge authenticates and rate-limits, the durable queue guarantees ordered at-least-once delivery, the worker pool runs your handler in isolation, and the result store keeps it queryable.

Persist first, run second — the rule the whole system is built around.



Live in *three moves*

Install

`npm i @nimbus/sdk` and drop your API key into the environment. No agents, no sidecars.

Define

Write a handler, give it a typed schema, and register it in `nimbus.toml`.

Submit

Call `nimbus.run()` from anywhere — the dashboard shows the job before the call even returns.

Three things you *stop worrying about*

Latency



Jobs start in under 50 ms
at p95 — fast enough to sit
in a request path.

Isolation



Each run is sandboxed;
one bad job can't take its
neighbours down.

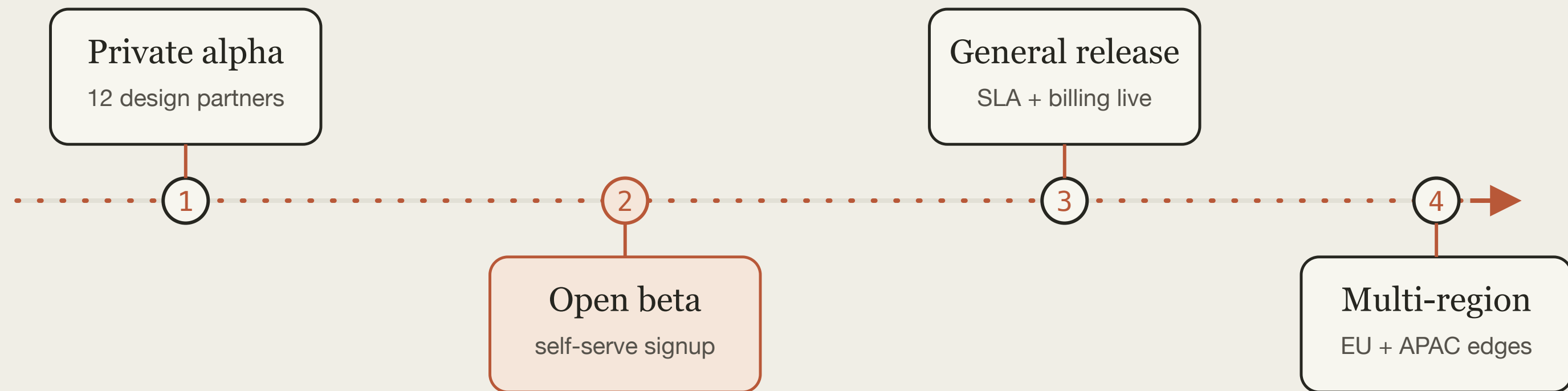
Reach



Execute close to your data
with edges in three
regions today, more on the
way.

• THE ROAD HERE AND AHEAD

A deliberate *rollout*



Open beta is live today; general release lands next quarter with billing and SLAs.

● IN THEIR WORDS

*We deleted **600 lines** of retry glue
the week we adopted Nimbus —
and stopped getting **paged** for it.*

• THE BET

Infrastructure should *disappear*

The best background job is one nobody thinks about. Nimbus is the boring layer that lets your team spend its attention on the product, not the plumbing.

• GET STARTED

Ship something *forgettable*

Open beta is free while it lasts. Bring a handler, leave the operations to us.